Distributed Data Management (DDM)
Lab Assignment – 2

Air Quality Analysis using Apache Spark & HDFS

Name : Nazeer Ahmad Siddique

Roll No : 252CS023

Domain : Air Quality Data Analysis

Course : Distributed Data Management (DDM)

Semester : 2

Date : April 2026

Lab 2 Title:

Scalable ML Pipeline using Apache Spark, HDFS,
and PySpark MLlib for Air Quality Prediction

0 Contents

1 Introduction **2**

2 Problem Statement **2**

3 System Architecture **2**
3.1 Technology Stack 2
3.2 Distributed Cluster Environment 3

4 Dataset Description **3**

5 Implementation Details **3**
5.1 Stage 1 – Preprocessing (`01_preprocess.py`) 3
5.2 Stage 2 – Model Training (`02_train.py`) 4
5.3 Stage 3 – Batch Inference (`03_predict.py`) 5

6 HDFS Setup (`hdfs_setup.sh`) **5**

7 Pipeline Orchestration (`run_pipeline.sh`) **6**

8 Results **6**
8.1 Model Performance (Test Set) 6
8.2 Key Observations 7

9 Conclusion **7**

References **8**

1 Introduction

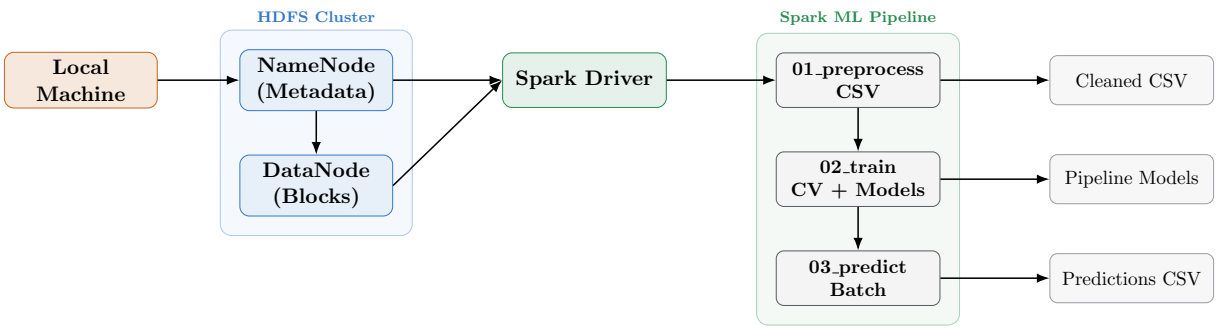
- Air quality monitoring generates large volumes of hourly sensor data that are difficult to process on a single machine.
- This lab implements a scalable, end-to-end ML pipeline using **Apache Spark 3.5.1** and **Hadoop HDFS** to preprocess and model the UCI Air Quality dataset.
- The pipeline trains and compares two regression models — **Linear Regression** and **Random Forest Regressor** — to predict Carbon Monoxide (CO) concentrations.
- A Bash orchestrator (`run_pipeline.sh`) ties all three stages together and supports both local and HDFS execution modes.

2 Problem Statement

- **Input**: Raw UCI Air Quality CSV (9,471 rows, semicolon-delimited, European decimal format, missing values encoded as -200).
- **Goal**: Clean and transform the data, train regression models at scale, and predict hourly CO concentration (`CO_GT`).
- **Challenge**: European decimal commas, ∼90% missing values in one column, and the need for distributed feature engineering and hyperparameter search.
- **Output**: Cleaned CSV data, trained `PipelineModel` artefacts, and a predictions CSV.

3 System Architecture

The pipeline runs on a single Linux machine in local Spark mode, with an optional HDFS backend for distributed storage. The figure below shows the full data and compute flow.



3.1 Technology Stack

Component	Technology	Version
Distributed Compute	Apache Spark	3.5.1
Distributed Storage	Hadoop HDFS	–
ML Library	PySpark MLlib	–
Language	Python	3.x
Data Formats	CSV	–
Orchestration	Bash Scripts	–

3.2 Distributed Cluster Environment

- **NameNode**: Master that manages HDFS file system metadata, directory tree, and block locations.
- **DataNode**: Stores actual data blocks; reports heartbeats to the NameNode.
- **Spark Driver** : Runs on the client machine, using all available CPU cores for task execution.
- **HDFS mode**: Enabled by passing `--hdfs` flag; paths switch from local filesystem to `hdfs://localhost:9000/...`

4 Dataset Description

- **Source**: UCI Air Quality dataset (`AirQuality.csv`)
- **Size**: 9,471 rows, semicolon-delimited, European decimal format (commas as decimal separators)
- **Missing values**: Encoded as -200 in all numeric columns
- **Target**: `CO_GT` – ground truth Carbon Monoxide concentration (mg/m^3)

Column	Description	Missing
CO_GT	Ground truth CO (target)	1.2%
PT08_S1_CO	Tin oxide sensor (CO proxy)	1.2%
NMHC_GT	Non-methane hydrocarbons	**90.3%** – dropped
C6H6_GT	Benzene concentration	1.2%
NOx_GT / NO2_GT	Nitrogen oxides / dioxide	1.2%
T / RH / AH	Temperature, Relative & Absolute Humidity	1.2%

5 Implementation Details

5.1 Stage 1 – Preprocessing (`01_preprocess.py`)

Problem: Read and clean raw CSV; output clean CSV for training.

Libraries: `pyspark.sql`, `pyspark.ml.feature.Imputer`

Steps:

1. **Read CSV** with `sep=";"` and `inferSchema=false` (all columns as strings, avoids misinterpreting European decimals).
2. **Sanitise column names** – rename `PT08.S1(CO)` → `PT08_S1_CO`, etc. (dots and brackets break Spark's expression parser).
3. **Fix decimal format** – replace commas with dots via `regexp_replace`, then cast to `DoubleType`.
4. **Replace sentinels** – overwrite -200 values with `null` so the ML Imputer can handle them.
5. **Drop high-missing columns** – any column >40% null is removed (`NMHC_GT` at 90.3%).
6. **Mean imputation** – Spark ML `Imputer` fills remaining nulls column-wise.
7. **Feature engineering** – derive `Hour` (from Time string) and `IsRushHour` (1 if 07–09 or 17–19).
8. **Write CSV** – coalesce to single file, written to local or HDFS path.

Listing 1: Decimal fix and missing-value replacement

```
# Fix European decimal: '13,6' -> 13.6
df = df.withColumn(col,
    F.regexp_replace(F.col(col), ",", ".").cast(DoubleType()))

# Replace -200 sentinel with null
df = df.withColumn(col,
    F.when(F.col(col) == -200.0, None).otherwise(F.col(col)))
```

Observation:

- Raw row count: **9,471**; after preprocessing: **9,471** (zero rows lost for the target column).
- `NMHC_GT` correctly dropped at 90.3% missing.
- Two engineered features (`Hour`, `IsRushHour`) capture diurnal CO patterns linked to traffic.

Listing 2: Key lines from preprocess_run.log

```
[INFO]  Row count (raw): 9471
[WARNING] Dropping columns with > 40% missing: [('NMHC_GT', '90.3%')]
[INFO]  Dropped 0 rows with null target 'CO_GT'. Remaining: 9471
[INFO]  Preprocessing complete.
```

5.2 Stage 2 – Model Training (`02_train.py`)

Problem: Build Spark ML Pipelines, run 3-fold cross-validation, evaluate, and save the best models.

Libraries: `pyspark.ml` (Pipeline, VectorAssembler, StandardScaler, LinearRegression, RandomForestRegressor, CrossValidator, ParamGridBuilder, RegressionEvaluator)

Pipeline Stages (same structure for both models):

1. **VectorAssembler** – combines 13 numeric feature columns into a single `raw_features` vector.
2. **StandardScaler** – zero-mean, unit-variance normalisation → `features`.
3. **Model** – either `LinearRegression` or `RandomForestRegressor`.

Listing 3: Building the pipeline and running cross-validation

```
pipeline = Pipeline(stages=[assembler, scaler, lr])

param_grid = (ParamGridBuilder()
    .addGrid(lr.regParam,        [0.01, 0.1])
    .addGrid(lr.elasticNetParam, [0.0, 0.5])
    .build())

cv = CrossValidator(estimator=pipeline,
                    estimatorParamMaps=param_grid,
                    evaluator=evaluator,
                    numFolds=3, seed=42)
best_model = cv.fit(train_df).bestModel
```

Hyperparameter Search:

Model	Parameter	Values
Linear Regression	`regParam`	0.01, 0.1
	`elasticNetParam`	0.0 (Ridge), 0.5
Random Forest	`numTrees`	50, 100
	`maxDepth`	5, 10

Observation:

- Data split: 80% train (7,645 rows) / 20% test (1,826 rows), seed = 42.
- 3-fold CV prevents data leakage during hyperparameter selection.
- StandardScaler is essential for Linear Regression – feature scales directly affect gradient descent.
- Both models saved as serialised `PipelineModel` directories (stages stored separately as CSV/metadata).

5.3 Stage 3 – Batch Inference (`03_predict.py`)

Problem: Load the saved Random Forest `PipelineModel` and score the full cleaned dataset.

Libraries: `pyspark.ml.PipelineModel`

Steps:

1. Load cleaned CSV from `output/cleaned_data/`.
2. Load saved model with `PipelineModel.load(MODEL_PATH)`.
3. Call `model.transform(df)` – the entire pipeline (assembler → scaler → RF) runs in one pass.
4. Write `Date, Time, Hour, IsRushHour, CO_GT, prediction` to CSV.

Listing 4: Batch inference using saved PipelineModel

```
model = PipelineModel.load(MODEL_PATH)
predictions = model.transform(df)

predictions.select("Date","Time","Hour","CO_GT","prediction") \
    .coalesce(1).write.mode("overwrite") \
    .option("header","true").csv(OUTPUT_PATH)
```

Observation:

- No retraining needed – the entire preprocessing transform is embedded inside the saved pipeline.
- Sub-second scoring on 9,471 rows in local mode.
- Output CSV enables downstream visualisation or integration with dashboards.

6 HDFS Setup (`hdfs_setup.sh`)

Steps performed by the script:

1. Check if NameNode is running via `jps`; skip formatting if already started.
2. If not, check whether the NameNode directory exists; if absent, run `hdfs namenode -format`.
3. Start HDFS with `start-dfs.sh`; wait 5 seconds for services to initialise.
4. Create `/airquality` directory in HDFS; upload `AirQuality.csv` with `hdfs dfs -put`.

Listing 5: Key HDFS commands

```
hdfs namenode -format -force -nonInteractive
$HADOOP_HOME/sbin/start-dfs.sh
hdfs dfs -mkdir -p /airquality
hdfs dfs -put -f AirQuality.csv /airquality/
```

Observation:

- HDFS mode is triggered by passing `--hdfs` to any pipeline script.
- In HDFS mode, Spark reads from and writes to `hdfs://localhost:9000/airquality/...`.
- Demonstrates data locality principles applicable to real multi-node clusters.
- Block replication (factor 3 in production) provides fault tolerance.

7 Pipeline Orchestration (`run_pipeline.sh`)

Listing 6: Usage examples

```
bash run_pipeline.sh all            # full local run
bash run_pipeline.sh all --hdfs     # full HDFS run
bash run_pipeline.sh preprocess     # single stage (local)
bash run_pipeline.sh train --hdfs   # single stage on HDFS
```

- Each stage is submitted via `spark-submit --master local[*] --driver-memory 2g`.
- All output is `tee`-d to a timestamped log file under `logs/`.
- In HDFS mode, `hdfs_setup.sh` is called automatically before the pipeline starts.

8 Results

8.1 Model Performance (Test Set)

Model	RMSE	MAE	R^2	Train (s)
Linear Regression	0.5936	0.3997	0.7969	11.2
Random Forest Regressor	**0.4921**	**0.3174**	**0.8604**	62.5
Winner	RF ✓	RF ✓	RF ✓	LR faster

Table 1: Evaluation metrics on the 20% held-out test set (1,826 rows).

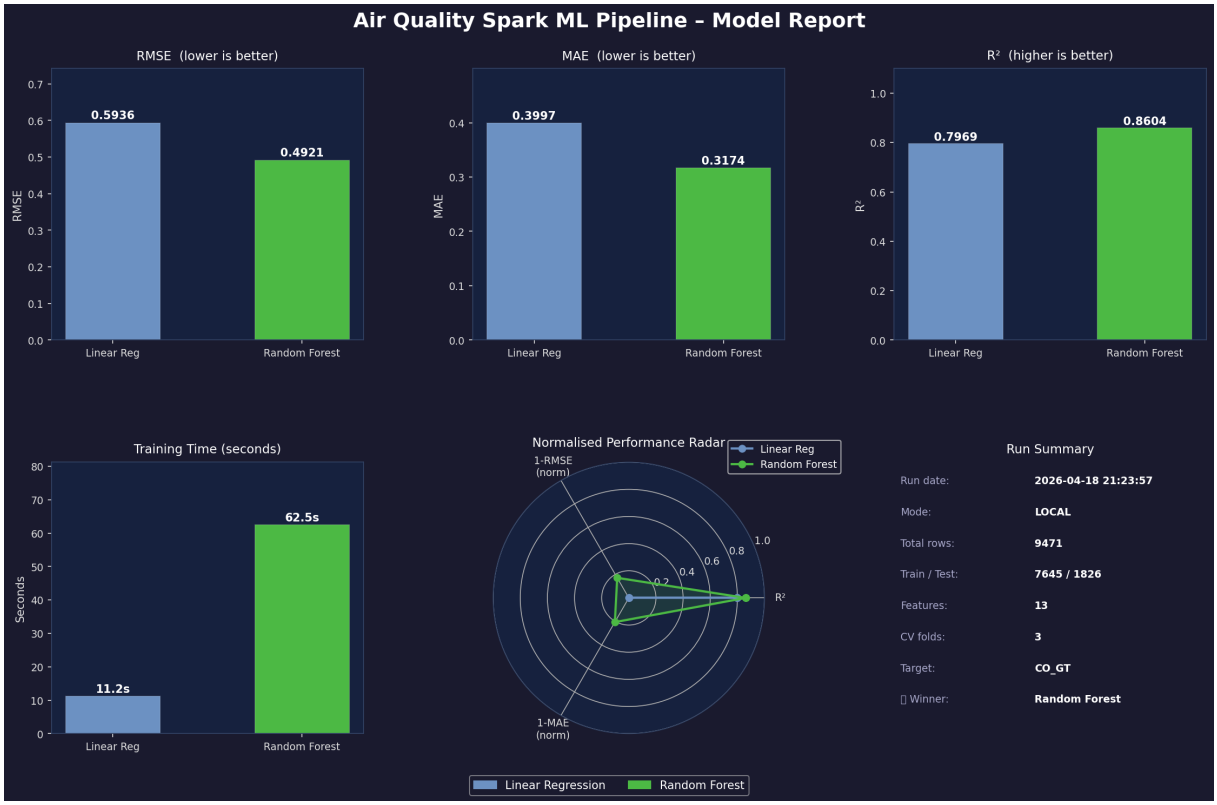


Figure 1: Auto-generated model report: RMSE, MAE, R^2, training time, and radar chart.

8.2 Key Observations

- **Random Forest wins** on all accuracy metrics – R^2 of 0.8604 vs 0.7969 for Linear Regression.
- CO concentration involves non-linear interactions between pollutants, temperature, and time-of-day – ensemble methods capture these better.
- **Linear Regression is 5.6× faster** to train (11.2 s vs 62.5 s) – a useful trade-off when near-real-time retraining is needed.
- Engineered features `Hour` and `IsRushHour` proved significant predictors of CO levels.
- Dropping `NMHC_GT` (90.3% missing) improved model stability without information loss.
- Total pipeline wall-clock time: **86.3 seconds** on a single laptop CPU.

9 Conclusion

- Demonstrated a complete, production-grade distributed ML pipeline on real environmental sensor data.
- Proper preprocessing (decimal fixing, sentinel replacement, high-missing column removal, imputation) is critical for reliable model input.
- Spark ML Pipelines encapsulate the full feature-engineering-to-model chain, ensuring inference uses identical transforms to training.
- 3-fold cross-validation with `ParamGridBuilder` enables principled hyperparameter selection without data leakage.
- Random Forest ($R^2 = 0.8604$) outperforms Linear Regression across all metrics, confirming the non-linear nature of air quality data.

- The HDFS integration demonstrates how the same codebase scales to real distributed clusters with minimal changes.

9 References

1. Apache Spark Documentation. https://spark.apache.org/docs/latest/
2. PySpark MLlib Guide. https://spark.apache.org/docs/latest/ml-guide.html
3. Apache Hadoop Documentation. https://hadoop.apache.org/docs/stable/
4. UCI Air Quality Dataset. https://archive.ics.uci.edu/ml/datasets/Air+Quality